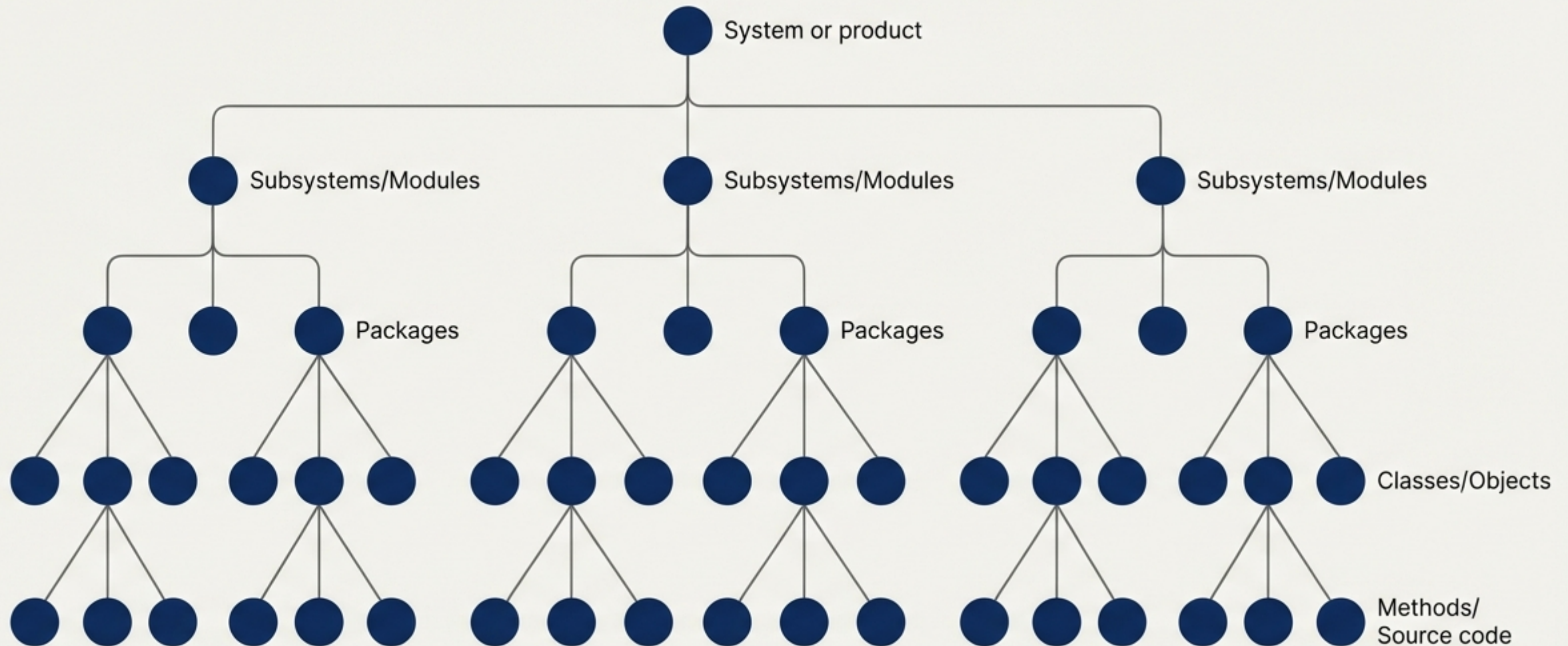
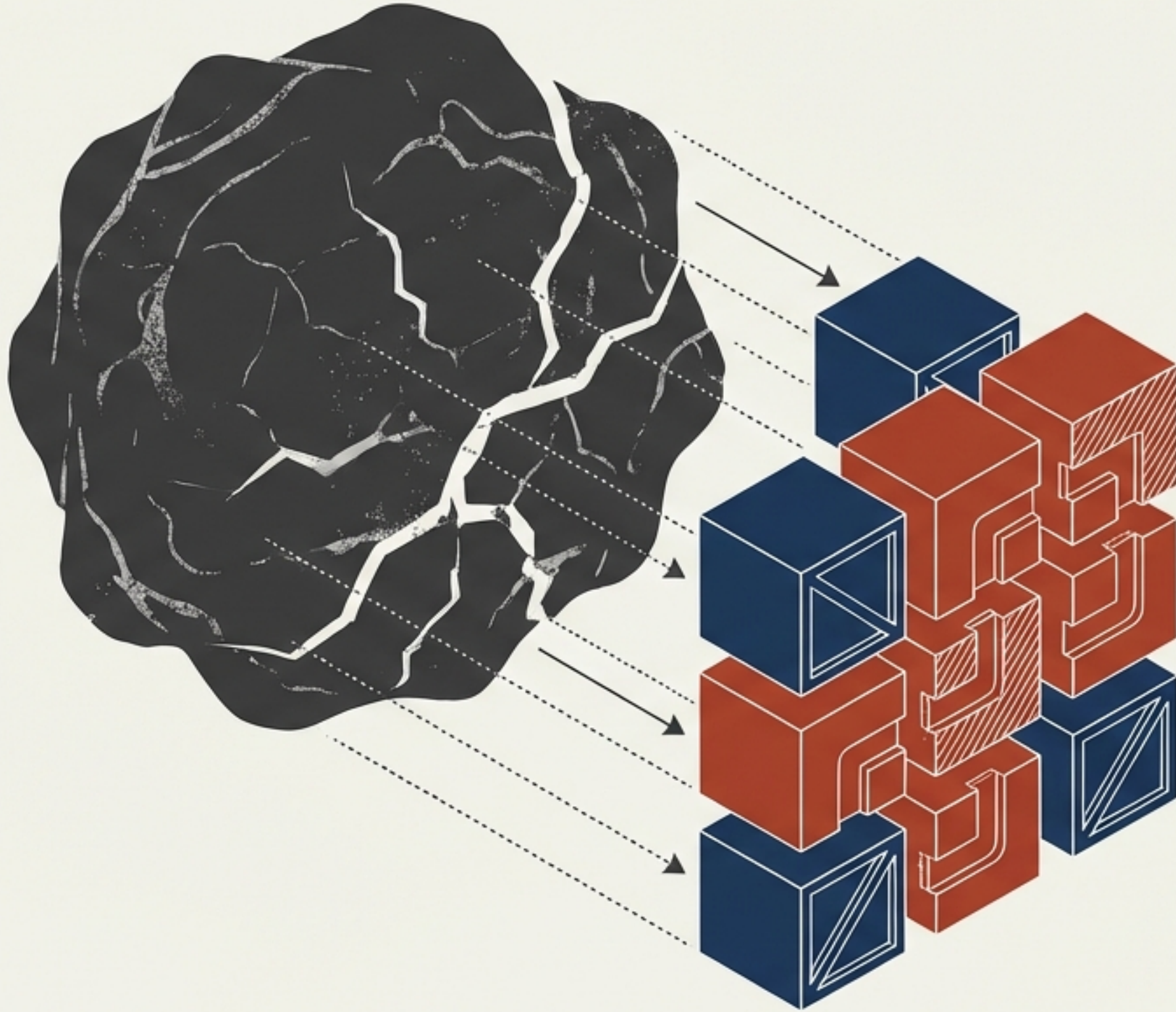


SOFTWARE IS NOT A LIST OF STATEMENTS; IT HAS A STRUCTURE.

How do we decide to break this apart?



DECOMPOSING TO CONQUER COMPLEXITY



- **Divide-and-conquer:** Tackle complexity in manageable pieces.
- **Reusability:** Identify and utilize existing parts.
- **Separation of concerns:** Decouple unrelated parts so they can evolve separately and flexibly.
- **Strategic focus:** Avoid reinventing the wheel to focus on the creative, unique elements of the system.

DEFINING SOFTWARE ARCHITECTURE

Architecture is a set of hard-to-change, high-level decisions that determine the structure of a solution.

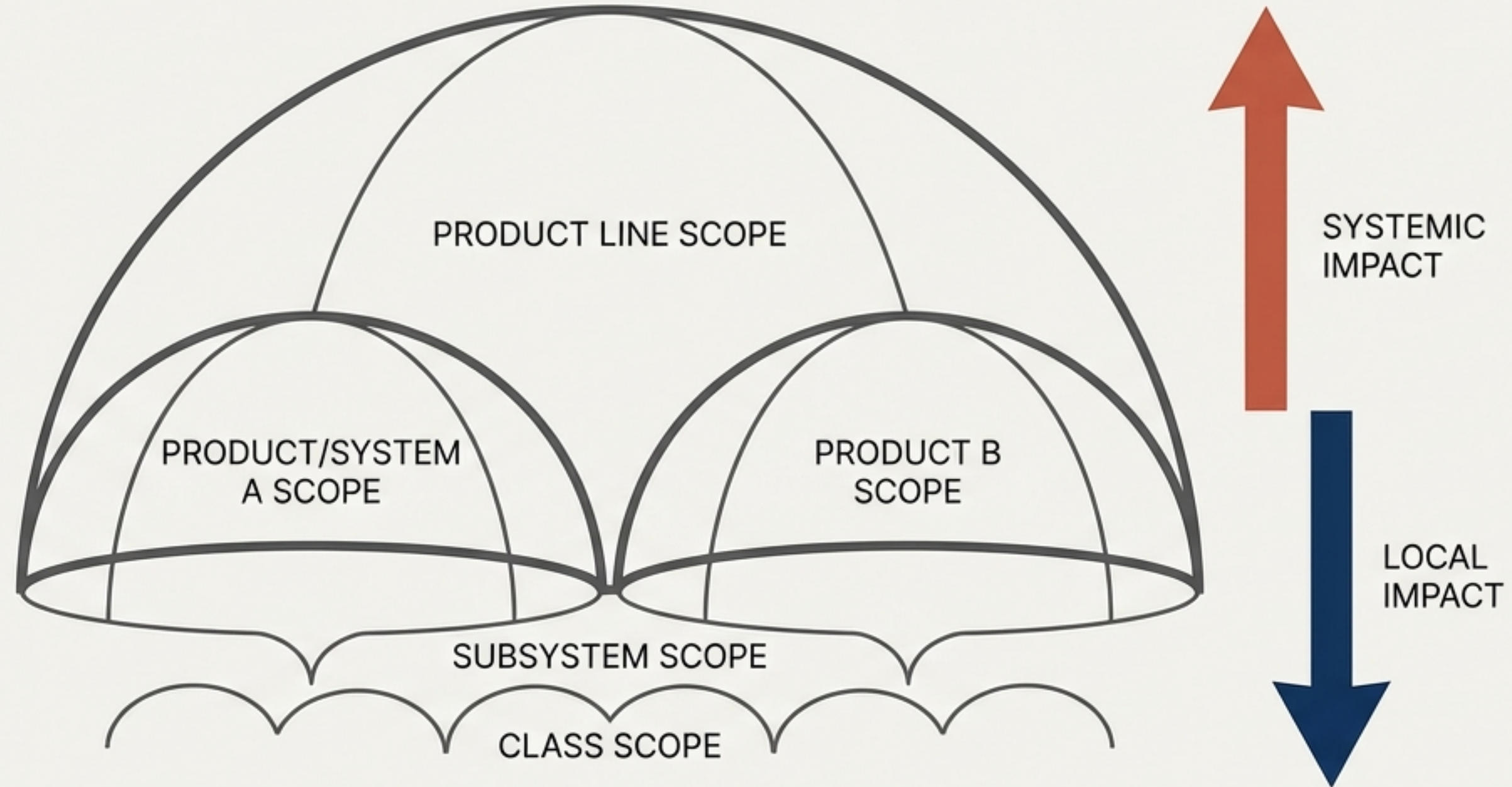
It dictates the parts of the system and their relationships.

It applies well-known solutions proven to work for similar problems.

Crucial mindset shift:

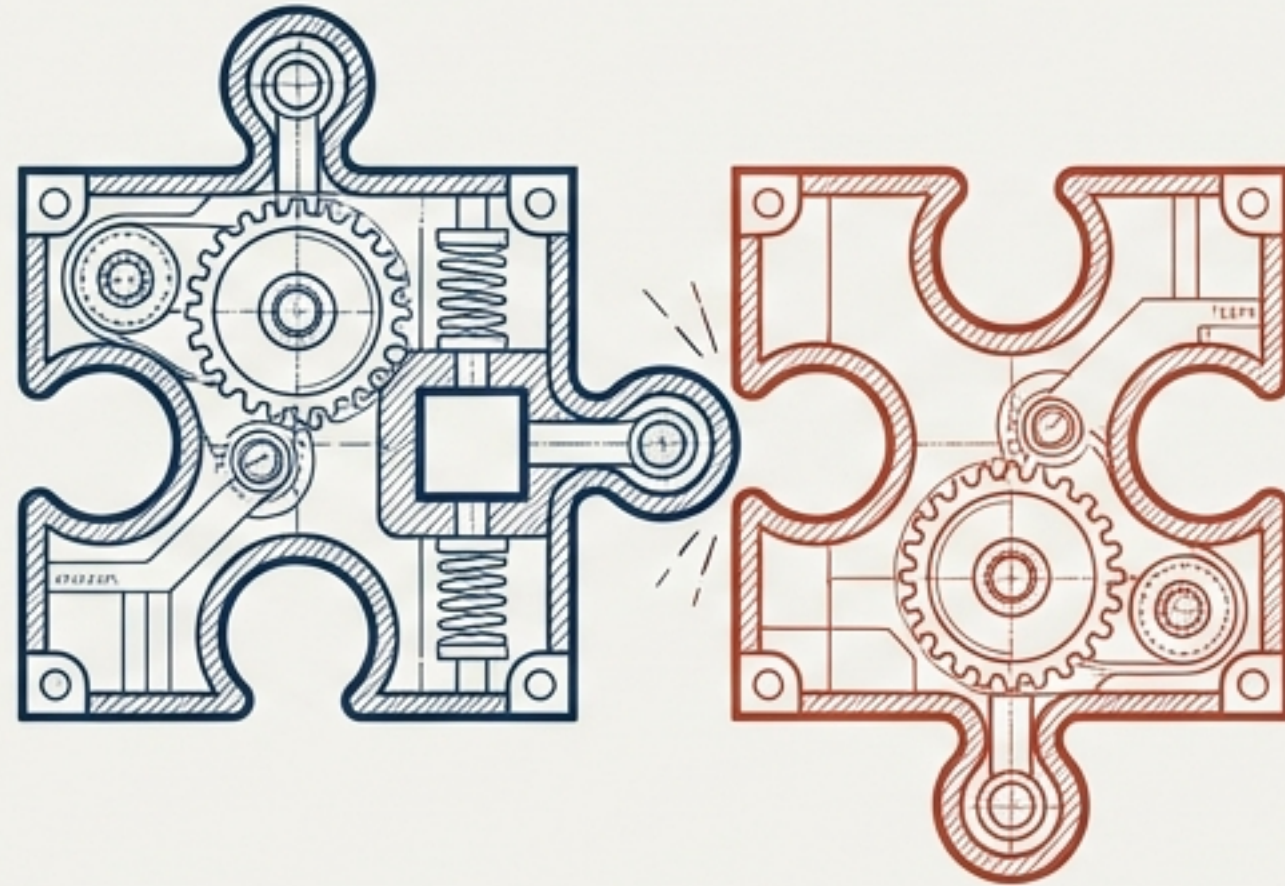
Software architecture is not a phase of development (like high-level design).
It is the principal design philosophy made early, affecting large parts of the system.

THE UMBRELLA OF SYSTEMIC IMPACT



A DECISION IS ARCHITECTURAL IF IT CAN BE MADE ONLY BY CONSIDERING THE PRESENT SCOPE.

THE ARCHITECT'S REALITY IS COMPROMISE



The best design for a component in isolation is rarely chosen when considering the broader context.

Goodness is subjective: Best for racing vs. Best for luxury.

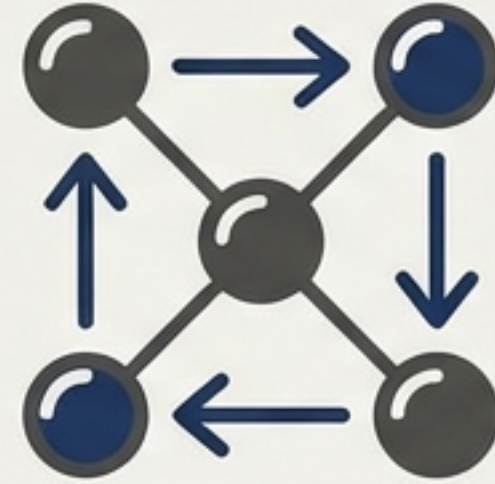
Architects must balance: Business priorities, available resources, competitor moves, technology trends, and backward compatibility.

LENSES OF CLARITY: DOCUMENTING ARCHITECTURE VIEWS



Module/Subsystem Views
(How it's built)

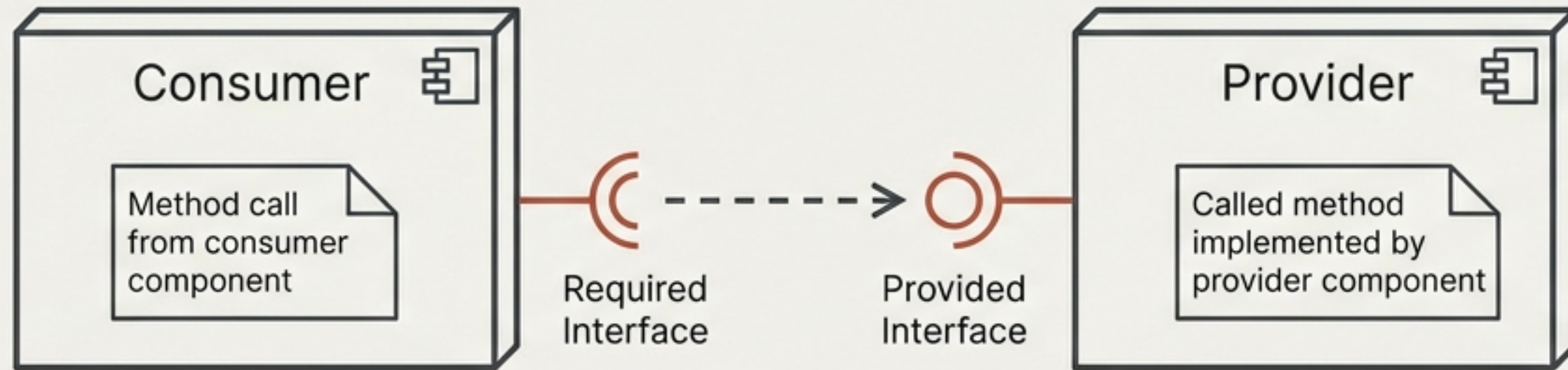
- Module Decomposition (Top-down refinement)
- Dependency View (How parts relate)
- Layered View
- Class View (Domain models)



Component & Connector Views
(How it runs)

- Process View (Sequence of activities)
- Concurrency View
- Shared Data View
- Client/Server View

THE LANGUAGE OF THE BLUEPRINTS

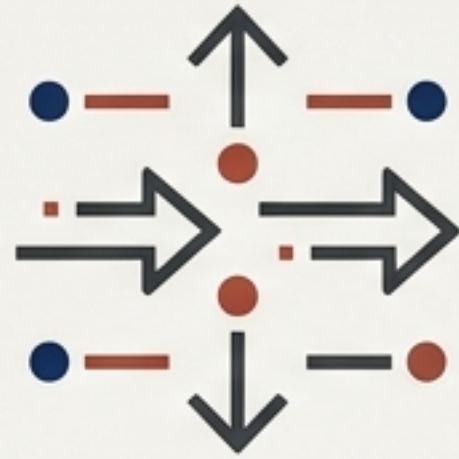


Behavior is defined entirely in terms of provided and required interfaces.

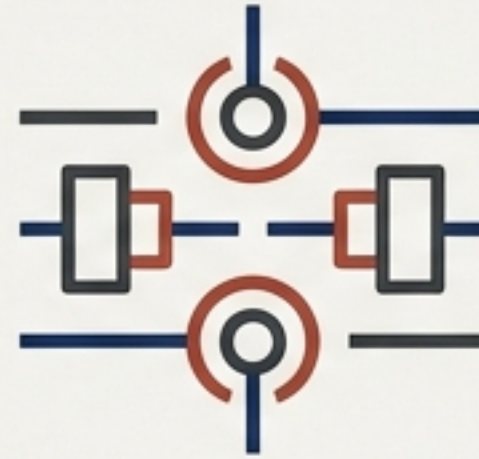
THE ANATOMY OF AN ARCHITECTURE STYLE



1. Components:
Processing elements that do the work.



2. Connectors: The enablers of communication (e.g., implicit events, explicit procedure calls, middleware).

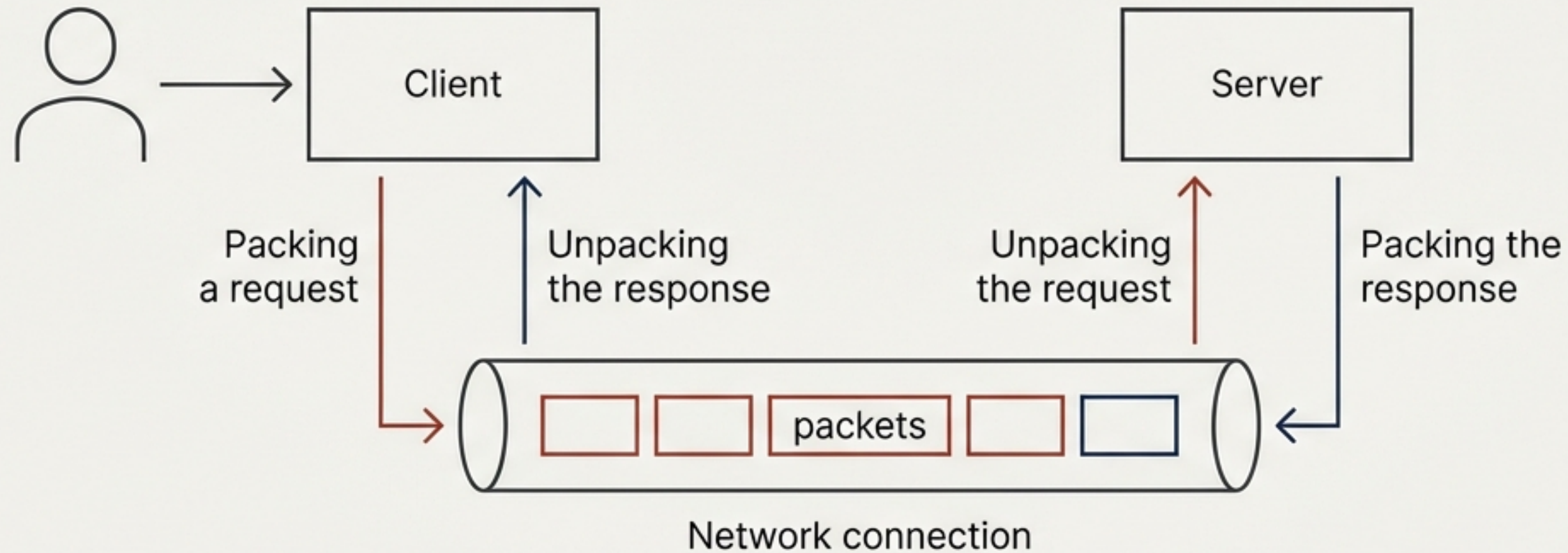


3. Interfaces:
Connection points dictating where data flows in and out.



4. Configurations: The specific topological arrangements of the components and connectors.

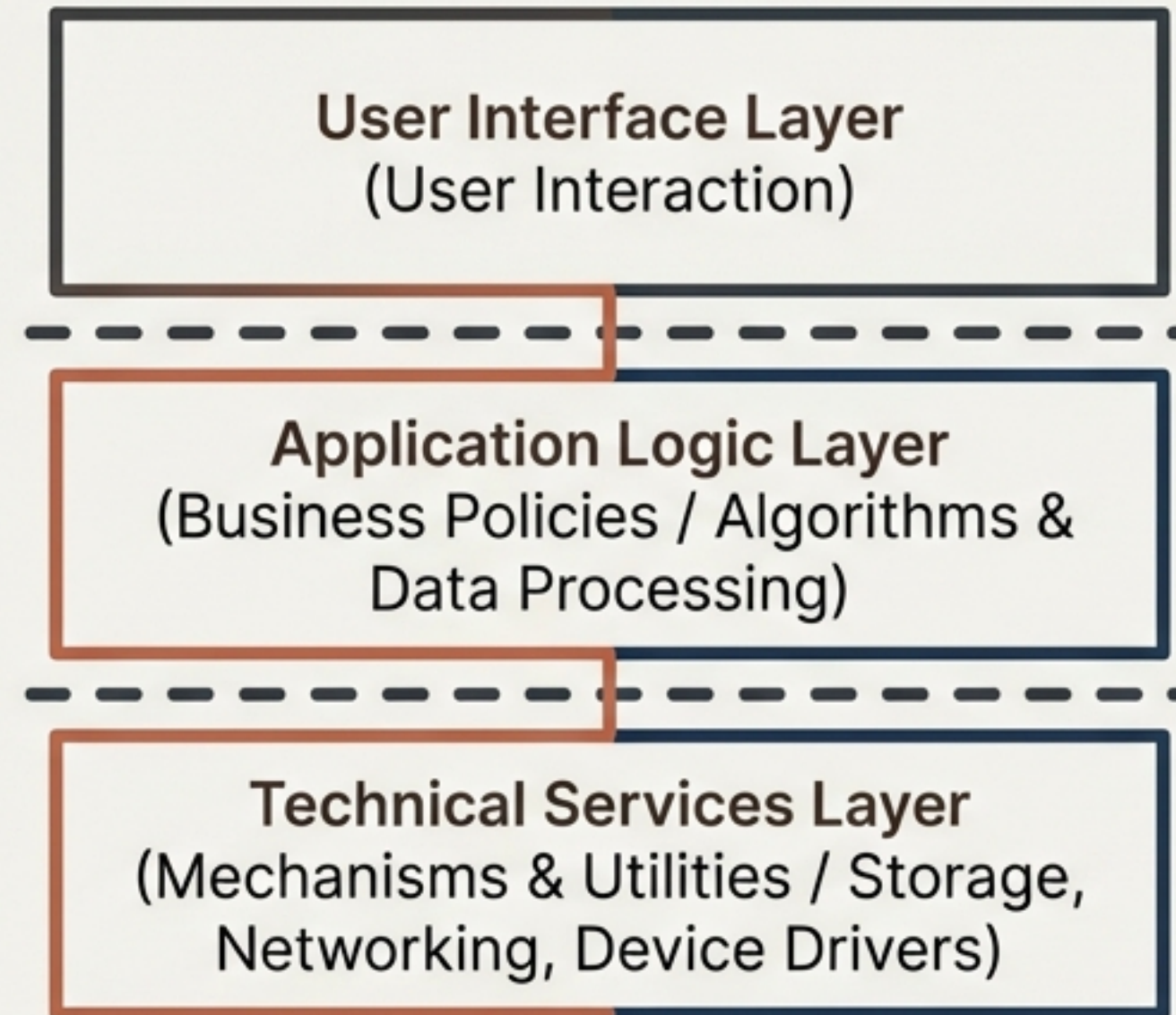
ARCHITECTURE STYLE: CLIENT/SERVER



Client: A triggering process that makes requests at times of its choosing.

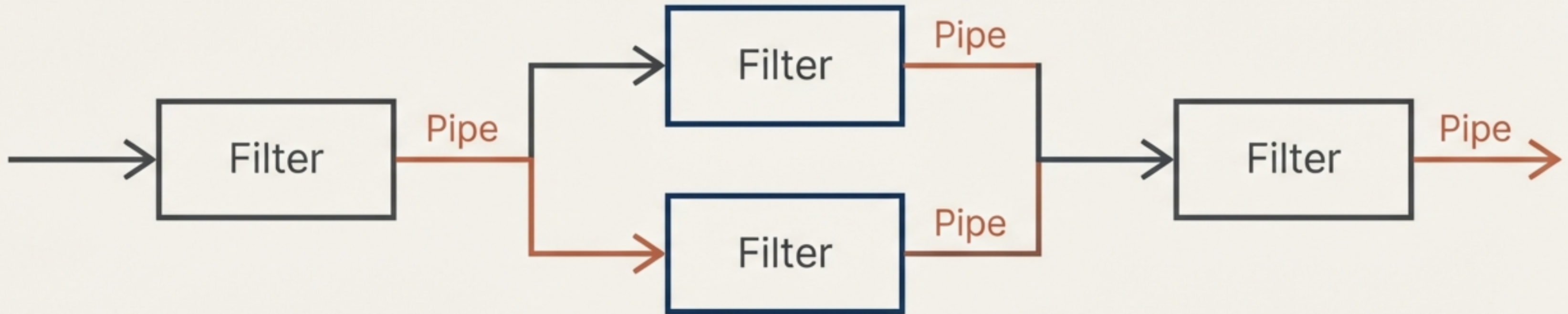
Server: A reactive process that listens, waits, and performs or rejects the request.

ARCHITECTURE STYLE: LAYERED (TIERED)



Organized hierarchically to reduce coupling. Inner layers are hidden from all except the adjacent outer layer, improving evolvability.

ARCHITECTURE STYLE: PIPE-AND-FILTER

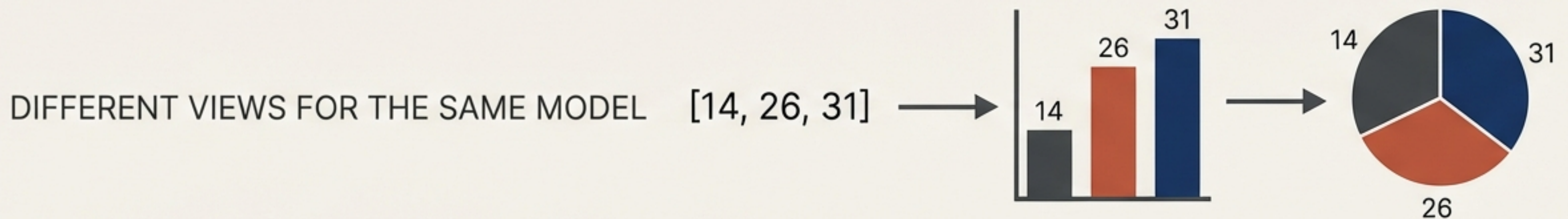
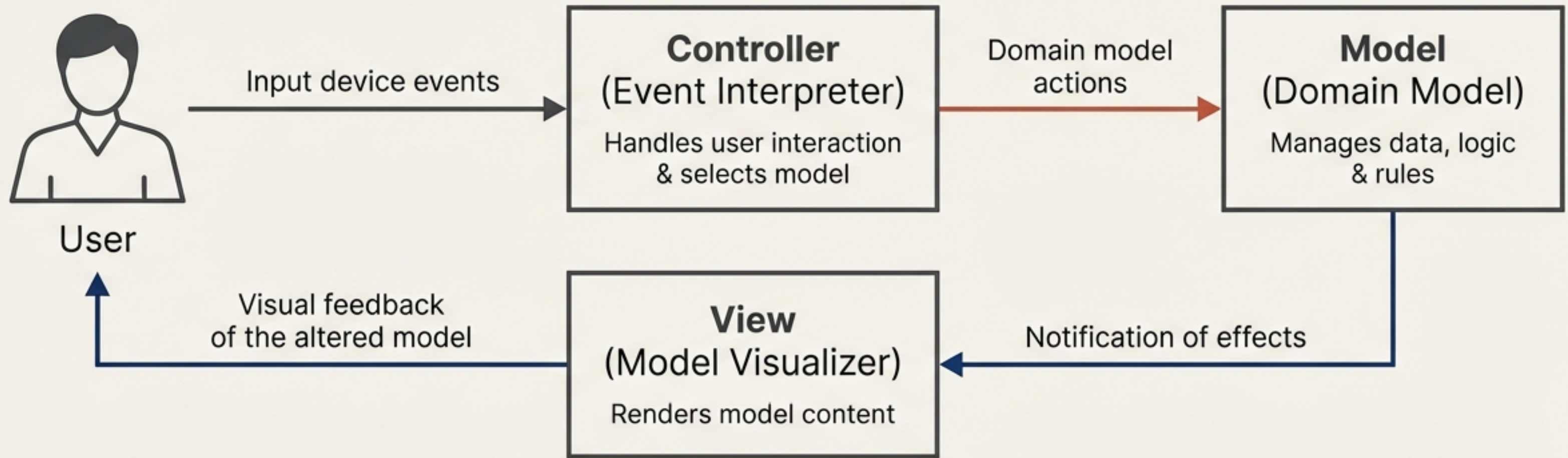


Components (Filters):
Transform input streams
into output streams

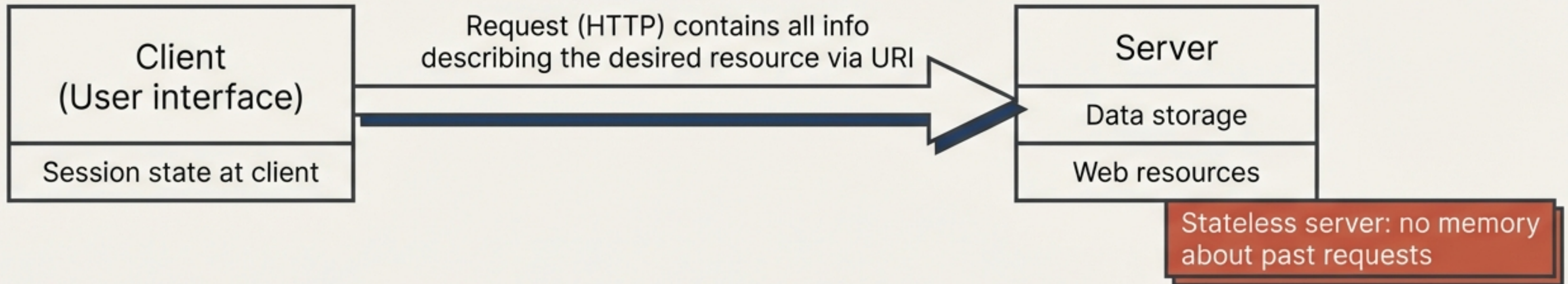
Connectors (Pipes):
Route data streams
between filters.

Classic Example:
UNIX shell scripts (`ls`
`| grep -v | more`)

ARCHITECTURE STYLE: MODEL-VIEW-CONTROLLER



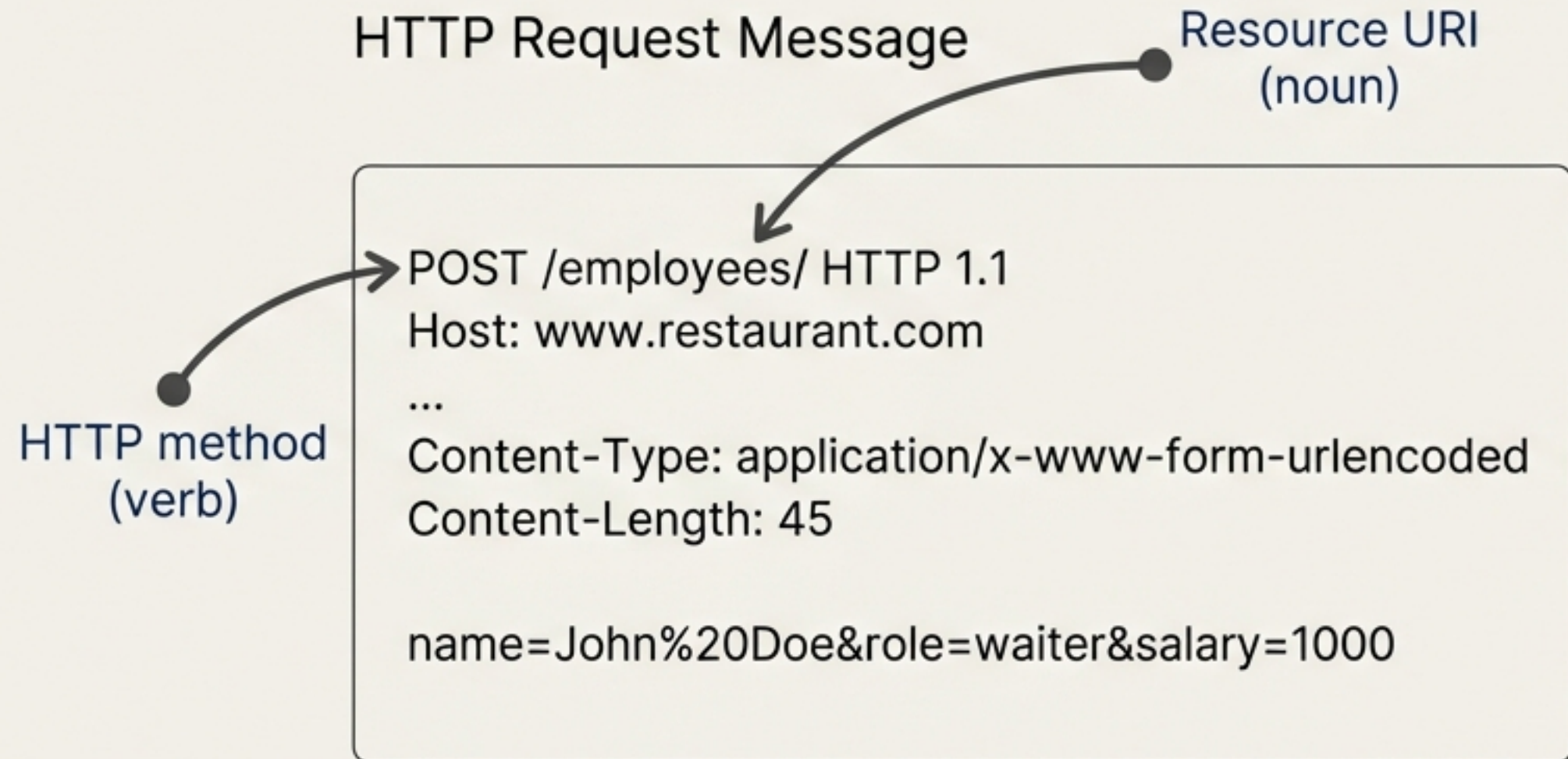
REST: A RESOURCE-BASED ARCHITECTURE



Representational State Transfer supports caching, uniform interfaces, and strict stateless sessions.

RESTFUL RESOURCE NAMING: NOUNS AND VERBS

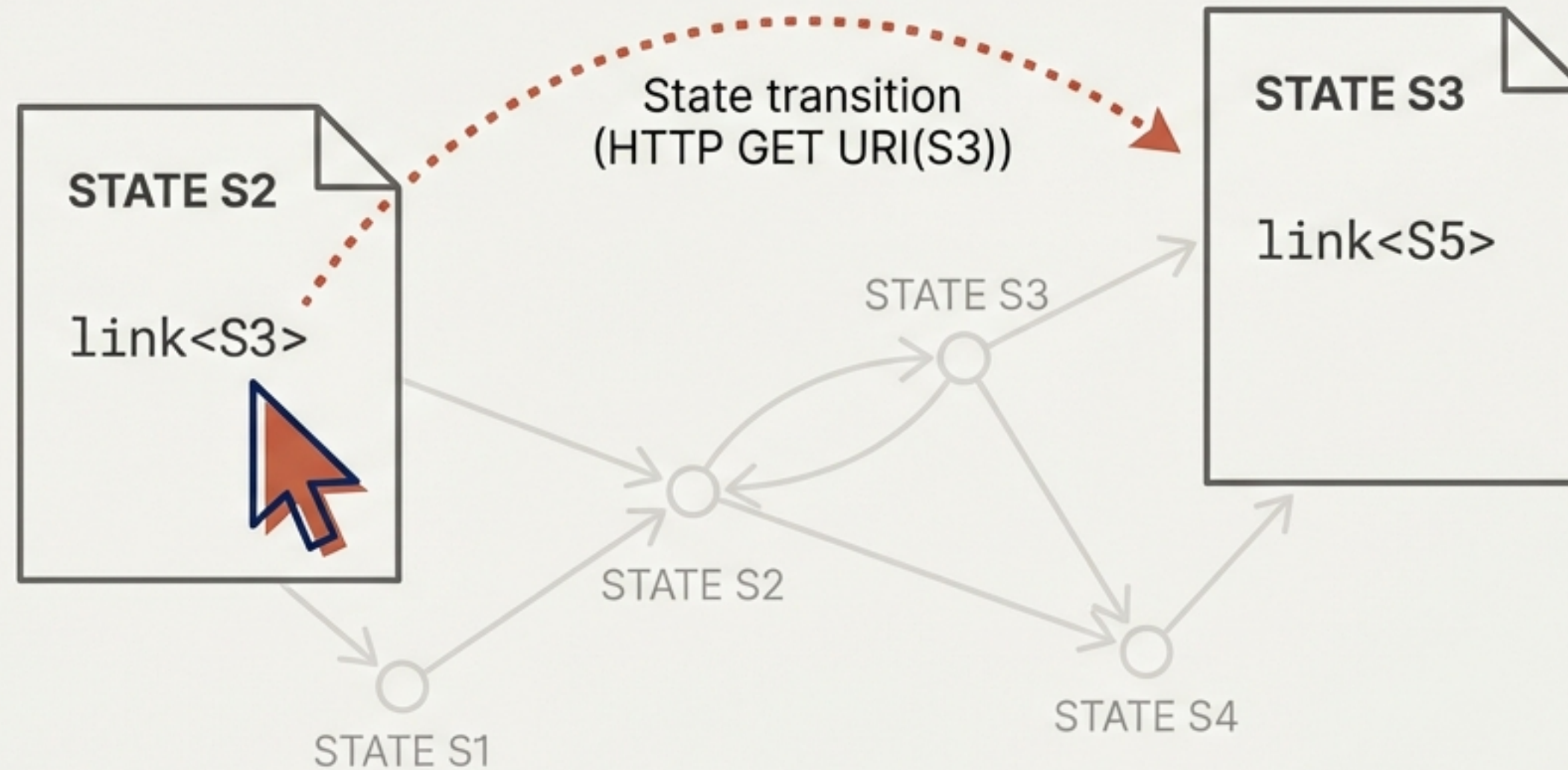
CRUD operation + Resource name + Input arguments
(verb) (noun)



The HTTP Protocol provides the connectors:

- POST: **Create** a new resource
- GET: **Read** a resource
- PUT: **Update** a resource
- DELETE: **Delete** a resource

THE ENGINE OF APPLICATION STATE (HATEOAS)

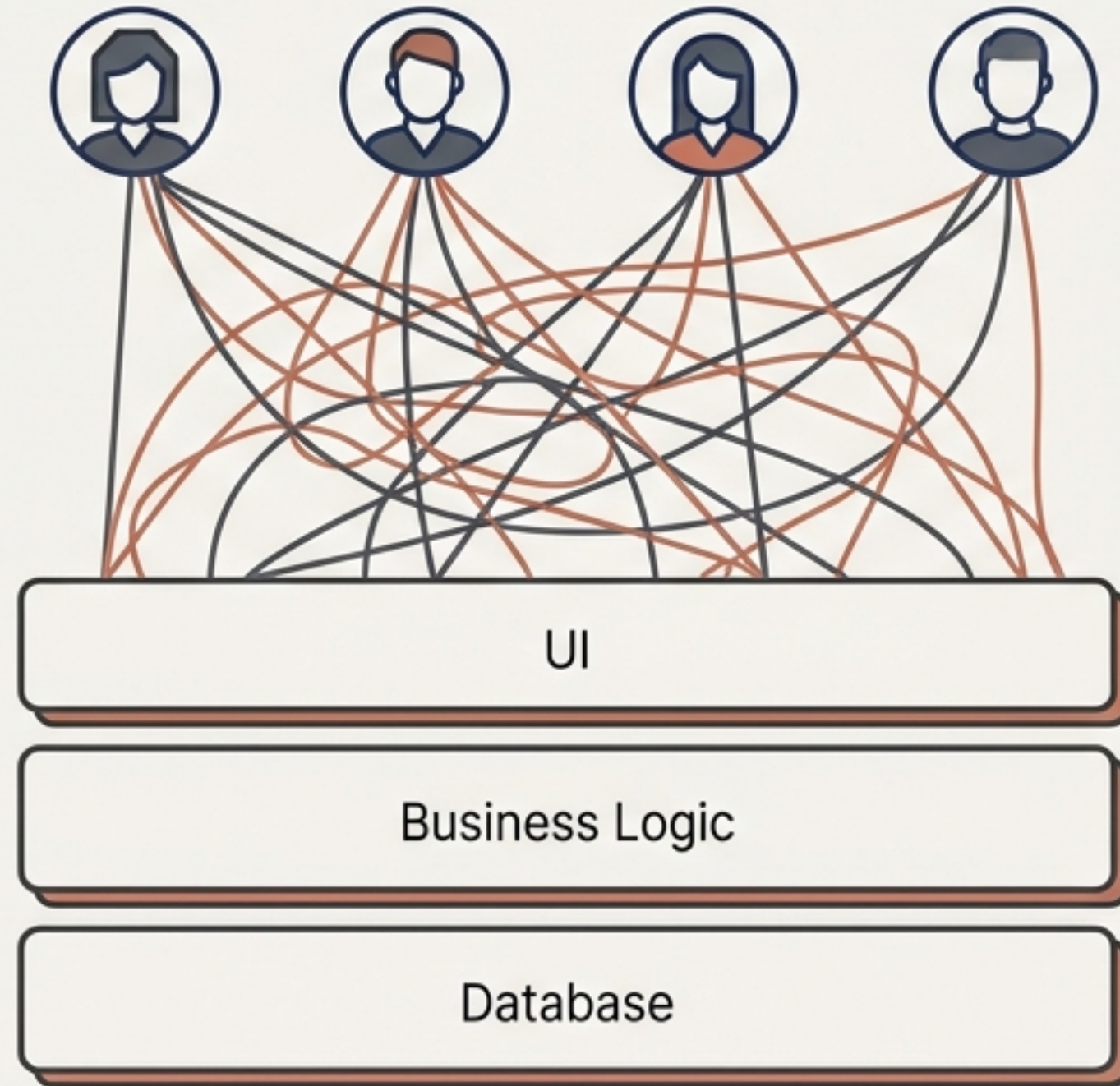


Hypermedia as the Engine of Application State.

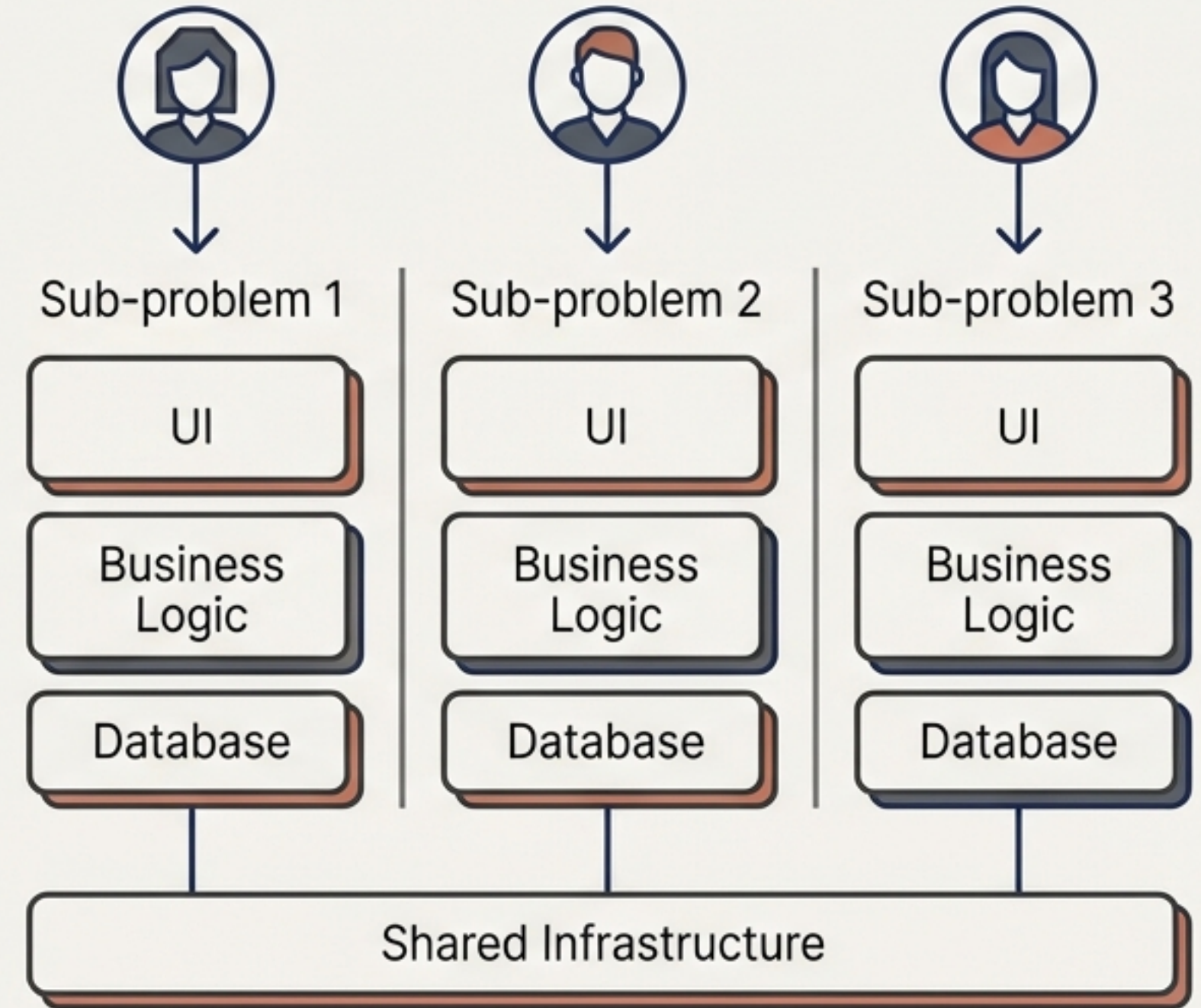
Hypertext contains embedded hyperlinks that dictate all possible valid transitions from the current state to the next.

MIRRORING REALITY: ORGANIZING THE WORK

SOLUTION-BASED

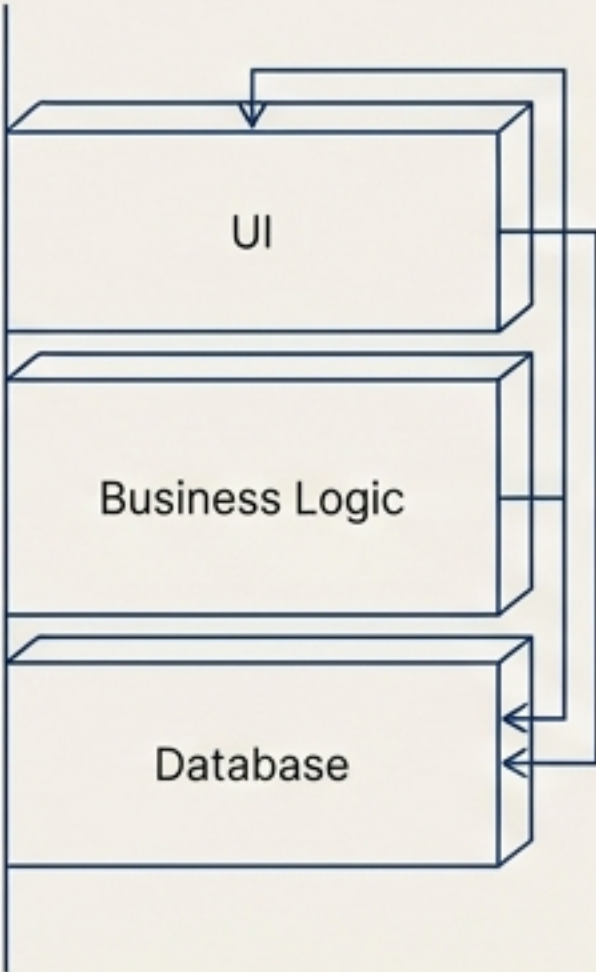
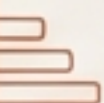


PROBLEM-BASED

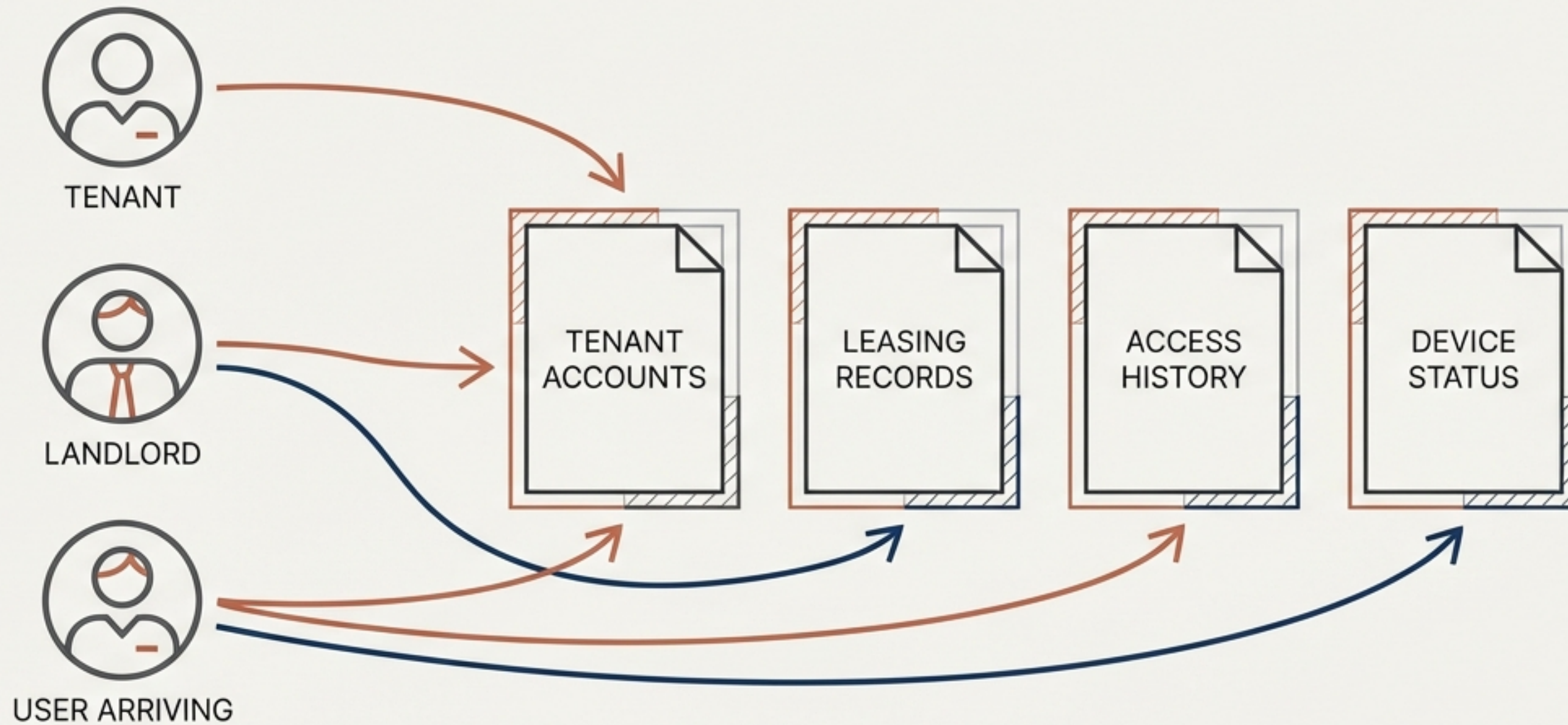


Conway's Law: Developer communication patterns and system module communication patterns will inevitably mirror each other.

DIVIDE BY PROBLEM VS. DIVIDE BY SOLUTION

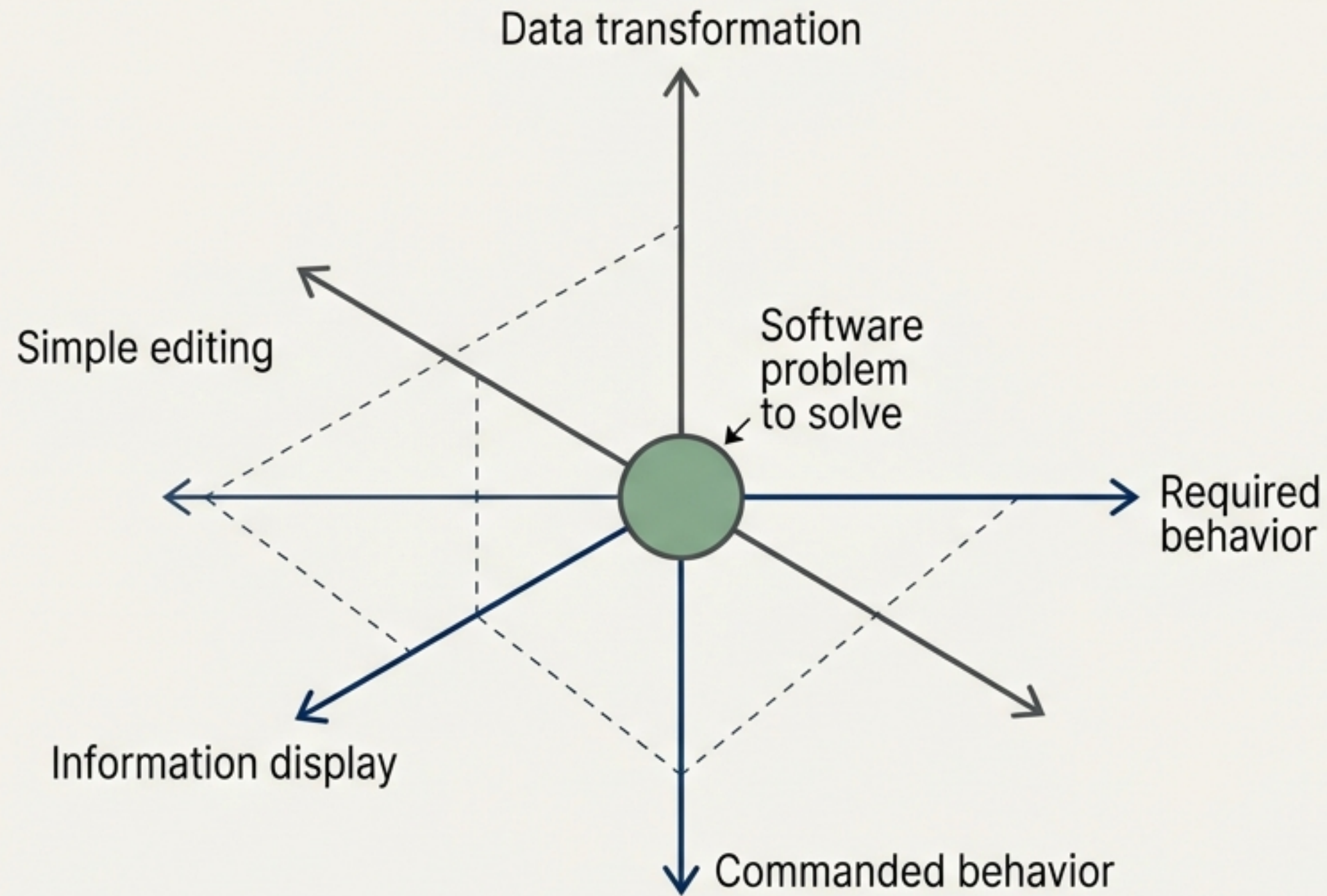
BY PROBLEM (VERTICAL SLICING):		BY SOLUTION (HORIZONTAL SLICING):	
	Organized around solving features/sub-problems.		Organized around technology layers (UI, DB). 
	Short feedback loops and mutually independent progress.		The original monolithic problem remains undivided. 
	Failure resilience: If one stack fails, the others can still succeed.		Long implementation time; high uncertainty integrating the parts later. 

IDENTIFYING NATURAL SUBPROBLEMS



Rationale: If the developer team's structure mirrors the product structure, the product structure must mirror the user contexts it supports.

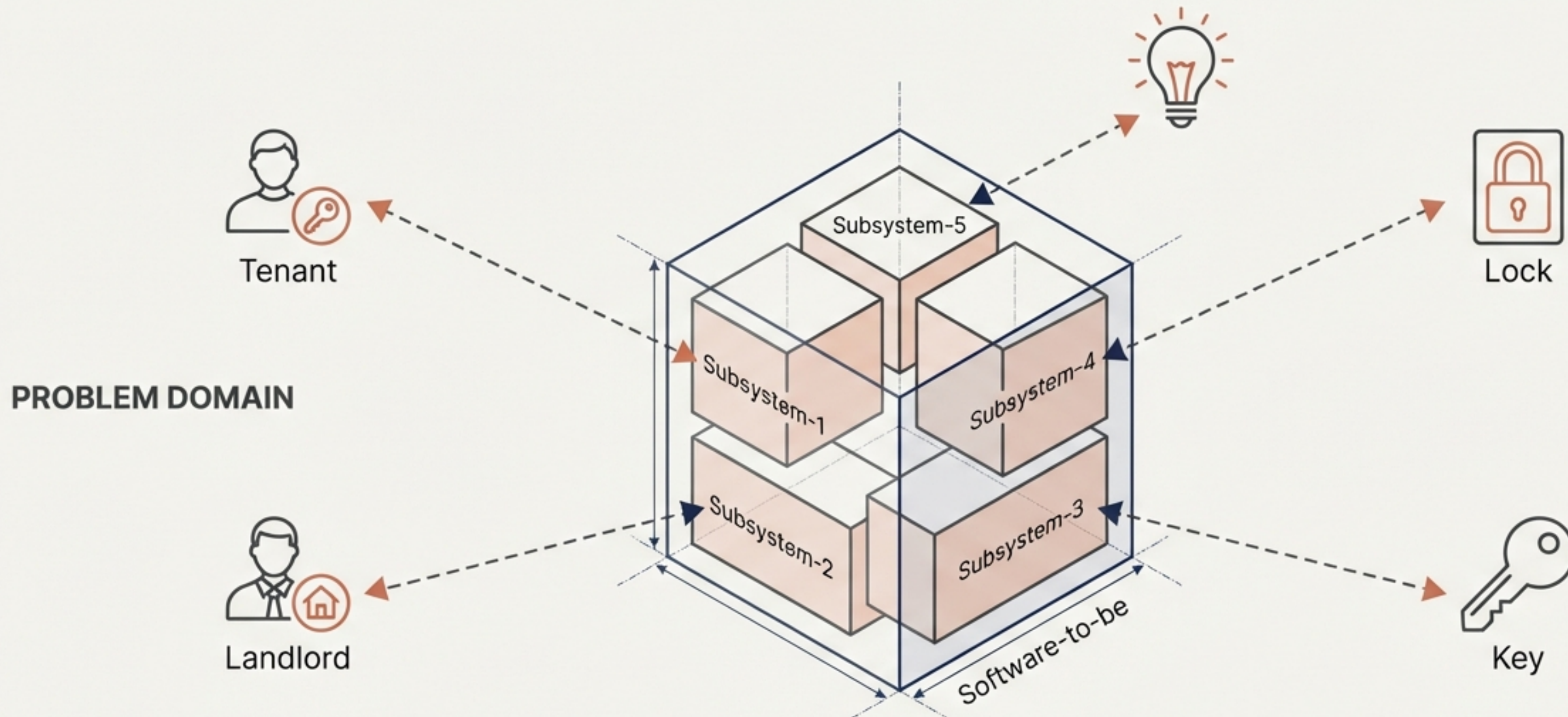
THE 5-DIMENSIONAL PROBLEM SPACE



The elementary problem types represent the coordinate system of the problem space.

Each subproblem can be analyzed independently and eventually recombined into the whole.

ALIGNING PROBLEM STRUCTURE WITH SOLUTION STRUCTURE



The ultimate goal: The structure of the solution must be carefully selected to perfectly fit the structure of the problem.